



PARAGON CORP

QUANTUM INITIATIVE · PARATECH ACADEMY

ParaQ IDE

Technical Specification

*A Quantum–Classical Integrated
Development Environment for ParagonCorp*

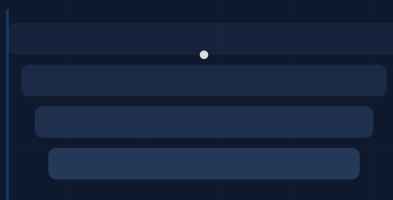
A Zed-based, Tauri v2–packaged IDE that natively integrates the
Quantinuum software stack (Guppy · HUGR · TKET · Selene) with an
agentic AI layer for physics, chemistry & biology in beauty,
cosmetics & healthcare

Zed / GPUI

Tauri v2

Quantinuum Stack

Agentic AI



Authored by **ParaTech Academy** · ParagonCorp Quantum Initiative

Abstract

ParaQ is an enterprise-grade Quantum–Classical Integrated Development Environment (QC-IDE) for ParagonCorp’s Quantum Computing and AI engineers. It is built as a fork of the Zed editor (Rust, GPUI) packaged as a desktop application with Tauri v2, and it integrates the open-source Quantinuum software stack — the Guppy quantum–classical language, the HUGR intermediate representation, the TKET/tket2 compiler and the Selene emulator — as first-class citizens of the editing, language-service and execution experience. Layered on top is an agentic AI subsystem that turns arXiv papers, book chapters, structured prompts and molecular inputs (SMILES/PDB/.xyz) into annotated, type-checked and emulated Guppy programs, orchestrating domain foundation models for chemistry, biology and physics through a sidecar inference service. This document is a complete, implementation-ready specification: it states the design goals, the layered architecture, detailed component contracts with typed Rust and Python signatures, data models and API schemas, the security and sandboxing model, performance targets and a benchmarking plan, a phased development roadmap, and a set of Architecture Decision Records that capture the rationale and the alternatives considered. Seven figures specify the system architecture, the agent workflow, the compilation pipeline, component interaction, the model-inference dataflow, the circuit-rendering pipeline and the end-to-end paper-to-simulation journey. The specification is intended to be directly actionable by a senior Rust/Python engineering team.

Document control	
Title	ParaQ IDE — Technical Specification
Author	ParaTech Academy (ParagonCorp Quantum Initiative)
Audience	Senior Rust/Python engineers; platform, ML and quantum-software teams
Edition	1.0 (internal, implementation-ready)
Status	Living document — revise as upstream (Zed, Tauri, Quantinuum) evolves
Compilation	pdfL ^A T _E X (self-contained; run 2–3× for cross-references)
Scope	Architecture, component contracts, APIs, security, performance, roadmap, ADRs

On versions and forward-looking claims

Every external dependency is pinned to a concrete version in Section 17. Upstream projects (Zed, Tauri, the Quantinuum stack and the foundation models) move quickly; treat all version numbers and capability claims as a snapshot and re-verify against the live sources in Section 20 before a procurement or release decision. Where a capability is aspirational rather than shipping today, the text says so explicitly.

Contents

Abstract	1
1 System Overview and Design Goals	4
1.1 What ParaQ is	4
1.2 Design goals	4
1.3 Primary personas and their surfaces	4
1.4 Non-goals	4
2 Architecture	4
2.1 Layer responsibilities	4
2.2 Process and trust topology	5

3	Zed Architecture Analysis	5
3.1	The shape of the workspace	5
3.2	GPUI: entities, views, elements	6
3.3	Buffers, language and the project model	6
3.4	The extension host: WASM and <code>zed_extension_api</code>	6
3.5	The agent stack and the Agent Client Protocol (ACP)	6
3.6	ADR	7
4	Component Specification: Zed Fork and GPUI Extensions	7
4.1	Modifications to the fork	7
4.2	GPUI view contract	7
4.3	ADR	8
5	Component Specification: Tauri v2 Shell and IPC	8
5.1	IPC design	8
5.2	JSON-RPC sidecar envelope	8
5.3	ADR	8
6	Component Specification: Guppy LSP and Tree-sitter Grammar	8
6.1	Tree-sitter grammar	9
6.2	Server capabilities and the linear-type analyzer	9
6.3	ADR	9
7	Component Specification: SELENE Emulation Integration	9
7.1	Emulation service contract	9
7.2	The ParagonNoise plugin	10
7.3	ADR	10
8	Component Specification: TKET Compiler Pipeline	10
8.1	Compile service contract	10
8.2	The <code>paragon.chem</code> extension	10
8.3	ADR	11
9	Component Specification: Agentic AI Subsystem	11
9.1	The agent state machine	11
9.2	Agent roles and tool schemas	11
9.3	Memory	11
9.4	Declarative experiments	11
9.5	LangGraph node signature	11
9.6	ADR	12
10	Component Specification: HuggingFace Inference Sidecar	12
10.1	Model registry and inference contract	12
10.2	Domain model coverage	13
11	Component Specification: Molecular I/O Pipeline	13
11.1	Pipeline stages and contract	13
11.2	ADR	13

12 Data Models and API Contracts	13
12.1 Tauri command surface	13
12.2 JSON-RPC method catalogue	13
12.3 Core Rust traits	14
12.4 OpenAPI surface for CI	14
13 Security and Sandboxing Model	14
13.1 Trust zones	14
13.2 Capability model	14
13.3 Sandboxing the sidecar	15
13.4 Prompt-injection and untrusted-content defence	15
14 Performance Targets and Benchmarking Plan	15
14.1 Latency and throughput budgets	15
14.2 Benchmarking harness	15
14.3 Reproducibility as a performance property	16
15 Component Interaction and Workflow	16
15.1 Component interaction (sequence)	16
15.2 Paper-to-simulation journey (swimlane)	16
16 Circuit and Graph Visualisation Pipeline	16
16.1 Rendering contract	16
17 Compatibility Matrix and Pinned Versions	17
17.1 Build and packaging	17
18 ParaQ Capability Development Plan	17
18.1 Capability-to-seam map	17
18.2 Workstream A — Guppy language extension (WASM)	18
18.3 Workstream B — Guppy LSP and HUGR-aware analysis	19
18.4 Workstream C — Selene emulation panel (fork crate)	19
18.5 Workstream D — HUGR and circuit GPU canvases (fork crates)	19
18.6 Workstream E — Athena as an ACP agent server	20
18.7 Workstream F — Document-to-circuit generation	20
18.8 Workstream G — Multimodal synthesis and HF inference	20
18.9 Workstream H — TKET compile integration	20
18.10 Sequencing and milestones	21
19 Development Roadmap	21
19.1 Risks and mitigations	21
20 References	23

1 System Overview and Design Goals

ParaQ exists because ParagonCorp’s quantum work spans three worlds that no single off-the-shelf tool unifies: a *quantum* world (Guppy programs, HUGR graphs, TKET passes, Selene emulation on Quantinuum hardware), a *scientific* world (chemistry, biology and physics of actives, fragrances, skin targets and formulations), and an *AI* world (domain foundation models and agentic orchestration). The IDE’s purpose is to make a single surface in which an engineer can read a paper, generate and type-check a quantum program, emulate it, interpret the result against a classical baseline, and write it up — without leaving the editor or hand-wiring the toolchain.

1.1 What ParaQ is

ParaQ is a desktop application. Its editor core is a fork of **Zed**, whose **GPUI** framework gives a GPU-accelerated, Rust-native text and UI runtime; its application shell is **Tauri v2**, which provides the cross-platform window, the secure capability model and the bridge to a Python sidecar. Into that shell **ParaQ** embeds the Quantinuum stack as native services: a **Guppy** language server, a **TKET/tket2** compilation service, a **Selene** emulation service and a **HUGR** graph viewer. An agentic AI layer — planner, retriever, code generator, validator and reporter — sits above these services and drives them on the engineer’s behalf.

1.2 Design goals

Table 1 states the goals that constrain every later decision. They are deliberately ranked: when two goals conflict, the higher one wins.

1.3 Primary personas and their surfaces

Three personas drive the requirements. The *quantum engineer* authors Guppy kernels and reads HUGR/TKET output; they live in the editor, the graph viewer and the emulation panel. The *computational scientist* thinks in molecules and literature; they live in the agent chat, the molecular viewer and generated notebooks. The *platform engineer* wires **ParaQ** into CI and manages model and stack versions; they live in the command palette, the configuration files and the benchmarking harness. Every feature below names which persona it serves.

1.4 Non-goals

ParaQ is not a hardware control system (it submits through Nexus; it does not drive ion traps), not a general-purpose chemistry suite (it orchestrates existing models and InQuanto rather than reimplementing them), and not an autonomous research agent (it always returns to a human gate before spending hardware credits or writing outside the workspace). Keeping these out of scope is what makes the in-scope surface buildable by a small

ParaQ design goals (ranked)

G1	Correctness-first	A quantum result is never trusted until type-checked, emulated and compared to a classical baseline; the IDE makes that path the default.
G2	Native quantum UX	Guppy/HUGR/TKET/Selene are first-class: completion, diagnostics, graph view and emulation feel built-in, not bolted-on.
G3	Agentic, not autonomous	AI accelerates the engineer; humans approve at defined gates and the agent never executes irreversible or paid actions alone.
G4	Open-stack, no fork drift	Adopt and <i>extend</i> the Quantinuum OSS through documented seams; pin versions; contribute fixes upstream rather than fork.
G5	Secure by construction	Tauri capabilities, sandboxed sidecars and least-privilege IPC; secrets never leave the OS keychain.
G6	Performance	Sub-frame editor latency, fast incremental LSP, and emulation/inference that run off the UI thread.
G7	Reproducibility	Seeds, versions and resource metrics are recorded for every emulation and run, so any result can be reproduced and audited.

Table 1. The ranked design goals. Higher-numbered goals yield to lower-numbered ones when they conflict (e.g. correctness over raw performance).

team.

2 Architecture

ParaQ is organised as seven layers (Fig. 1). Each layer depends only on the one beneath it, so any layer — most importantly the upstream Zed, Tauri and Quantinuum dependencies — can be upgraded without rewriting the others. The ParagonCorp-owned layers are the agentic AI subsystem, the integration services and a thin set of extensions; everything below is adopted and extended, never forked.

2.1 Layer responsibilities

L0 — Foundations. The upstream dependencies: the Quantinuum OSS (**guppylang**, **hugr**, **tket2**, **selene-sim**, **pytket**, Nexus client), the HuggingFace model ecosystem, and the execution targets (Selene emulators locally, Helios via Nexus). **ParaQ** treats this layer as a pinned, versioned dependency set (Section 17).

L1 — Editor core. A fork of Zed supplies the buffer model, multibuffers, the language registry, tree-sitter

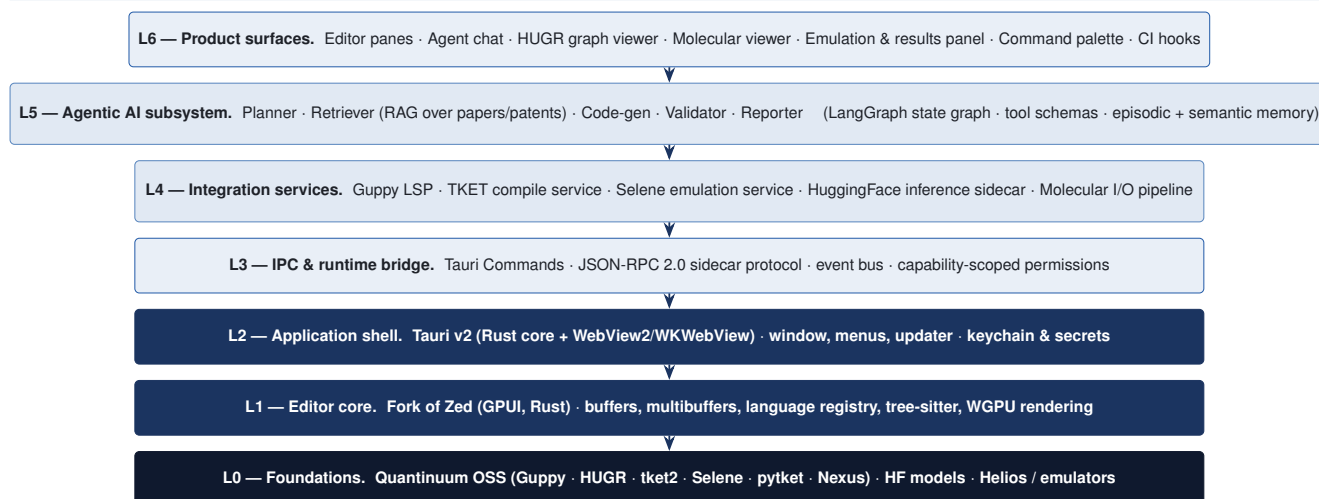


Figure 1. (D1) The seven-layer **ParaQ** architecture. ParagonCorp owns L4–L6 (integration services, the agentic subsystem and product surfaces); L0–L3 are adopted from Zed, Tauri and Quantinuum and extended through documented seams rather than forked. Control flows down; results and events flow up.

integration and the GPUI/WGPU rendering pipeline. **ParaQ**'s modifications are additive GPUI views and a language registration for Guppy (Section 4).

L2 — Application shell. Tauri v2 provides the cross-platform window, the auto-updater, the OS keychain bridge for secrets and — crucially — the capability system that scopes what the front-end and sidecars may do (Sections 5 and 13).

L3 — IPC and runtime bridge. Front-end (GPUI) to core (Rust) calls use Tauri Commands; core to Python sidecars use a JSON-RPC 2.0 protocol over stdio or a local socket. An internal event bus fans out progress, diagnostics and agent events to the UI.

L4 — Integration services. The five native services that make the quantum stack feel built-in: the Guppy LSP (Section 6), the TKET compile service (Section 8), the Selene emulation service (Section 7), the HuggingFace inference sidecar (Section 10) and the molecular I/O pipeline (Section 11).

L5 — Agentic AI subsystem. A LangGraph state graph of cooperating agents (planner, retriever, code-gen, validator, reporter) with tool schemas and a two-tier memory (Section 9). This is the layer that turns a brief or a paper into a validated result.

L6 — Product surfaces. What the engineer touches: editor panes, the agent chat, the HUGR graph viewer, the molecular viewer, the emulation/results panel, the command palette and the CI hooks (Sections 16 and 19).

2.2 Process and trust topology

ParaQ runs as three trust zones in distinct processes. The *core process* (Rust: Zed fork + Tauri) is trusted and holds no model weights. The *Python sidecar* (AI runtime, TKET, Selene, HF inference) is a separate,

sandboxed process that holds no secrets and reaches the network only through an allowlist. The *web front-end* (GPUI-rendered; any embedded WebView content) is the least trusted and may call only the capability-scoped commands it has been granted. Section 13 makes this concrete; the key invariant is that untrusted content (a parsed PDF, a model completion, a web page) can never widen its own permissions.

The one architectural rule

ParaQ never forks the Quantinuum stack and never patches Zed or Tauri internals in place. L4–L6 only *add* services, GPUI views, LSP features, HUGR/TKET extensions and Selene plugins through documented interfaces. When a ParagonCorp extension proves generally useful, it is contributed upstream rather than carried as a private fork that drifts from releases (G4).

3 Zed Architecture Analysis

Persona: platform engineer. Because **ParaQ** is built on a fork of Zed, its design is constrained and enabled by Zed's real architecture. This section analyses that architecture from the upstream source tree (zed-industries/zed, ≈234 crates under crates/, 97.7% Rust) and the project's own engineering rules, then draws the consequences for **ParaQ**. Every claim here is anchored to a primary source in Section 20; version specifics are a snapshot and are pinned in Section 17.

3.1 The shape of the workspace

Zed is a single Cargo workspace of small, single-responsibility crates rather than one monolith. The pieces **ParaQ** touches most are the UI framework

(`gpui`), the editor and buffer model (`editor`, `text`, `language`, `multi_buffer`), the project and filesystem model (`project`, `worktree`, `fs`), the language-service layer (`lsp`, `languages`), the workspace/panel system (`workspace`), the extension host (`extension`, `extension_host`, `extension_api`), and the agent stack (`agent`, `agent_servers`, `acp_thread`). **ParaQ**'s own code lives in a parallel set of `crates/paraq_*` and `crates/paragon_*` crates that depend on these but never edit them in place (G4). The toolchain is pinned by `rust-toolchain.toml` to a recent stable Rust with `rust-analyzer`, `clippy` and `rustfmt`, and — tellingly for **ParaQ** — three extra targets: `wasm32-wasip2` (extensions), `wasm32-unknown-unknown` (GPU on the web) and `x86_64-unknown-linux-musl` (the remote server).

3.2 GPUI: entities, views, elements

GPUI is a hybrid immediate-and-retained-mode, GPU-accelerated UI framework that also provides Zed's state and concurrency primitives. It offers three *registers* that **ParaQ** uses directly. *Entities* (`Entity<T>`) are handles to application state owned by GPUI and reached through a context; they are how a Selene run, an agent session, or a molecule model lives in the app. *Views* are entities that implement the `Render` trait and build a tree of styled elements with a Tailwind-style API — this is how every **ParaQ** panel is drawn. *Elements* are the low-level, imperative building blocks for custom rendering such as the circuit and HUGR canvases. Everything starts with an **Application**, which opens windows and registers a root view; rendering is on Metal (macOS), and a WGSL/GPU path underlies the element pipeline (the repository ships WGSL shaders).

The concurrency model matters for **ParaQ**: long operations (emulation, model inference, compilation) must run on the `BackgroundExecutor` via `cx.spawn`, returning to the foreground only to update an entity, so the UI holds 120 fps while a 20-qubit statevector run proceeds (G6). Async contexts (`AsyncApp`, `AsyncWindowContext`) are held across await points and are exactly where **ParaQ**'s JSON-RPC calls to the sidecar live.

3.3 Buffers, language and the project model

Zed represents text with a rope-backed buffer (`text/buffer`) and multi-file editing through `multi_buffer`; syntax is Tree-sitter, driven by queries shipped per language. The `language` crate registers a language, its grammar, injections, and the `LspAdapter` that launches and speaks to a language server; `project/worktree/fs` model the open folders, file watching and remote access. For **ParaQ** the consequence is concrete: **Guppy support is registered exactly the way every Zed language is**, and the HUGR/circuit views attach to buffers and projects through these existing APIs rather than a parallel universe.

Concept	Type	Role for ParaQ
Application	<code>Application</code>	Process root; opens windows, owns globals.
Context	<code>App / Context<T></code>	Read/update entities and globals; <code>cx</code> everywhere.
Entity	<code>Entity<T></code>	Owns a Selene run, agent session, molecule, circuit model.
View	<code>impl Render</code>	Each ParaQ panel (emulation, HUGR, molecule, agent).
Element	<code>impl Element</code>	Circuit/HUGR canvas, custom list virtualisation.
Action	<code>#[action]</code>	Palette commands (<code>paraq: emulate</code> , etc.).
Executor	<code>BackgroundExecutor</code>	Off-thread emulation/inference; timers in tests.

Table 2. The GPUI concepts **ParaQ** builds on, drawn from Zed's framework and engineering rules. The context object `cx` threads through every callback; windows are passed as window before `cx`.

3.4 The extension host: WASM and `zed_extension_api`

Zed has a first-class extension system. An extension is a Git repository with an `extension.toml` manifest; procedural logic is written in Rust, compiled to `wasm32-wasip2` as a `cdylib`, and links the `zed_extension_api` crate, implementing the `Extension` trait and registering with `register_extension!`. Extensions can ship *languages* (a `config.toml` plus Tree-sitter `highlights.scm` and friends), *themes*, *icon themes*, *snippets*, *debuggers*, and *MCP servers* — and, historically, *agent servers*. Crucially, the WASM sandbox is restricted: `std::env::var` and `cfg` do not behave as in native code, and host access goes through the API's `Worktree` and `current_platform` surfaces. This draws a sharp line for **ParaQ** (Fig. 2): *anything expressible as a language/theme/MCP-server extension should ship as a versioned WASM extension* (no fork), while *panels*, *custom GPU canvases* and *deep agent integration* require fork-level GPUI crates.

3.5 The agent stack and the Agent Client Protocol (ACP)

This is the most strategically important discovery of the source analysis. Zed ships a native agent (`agent`) and a protocol for external agents: the **Agent Client Protocol** (ACP), a bidirectional, JSON-RPC-

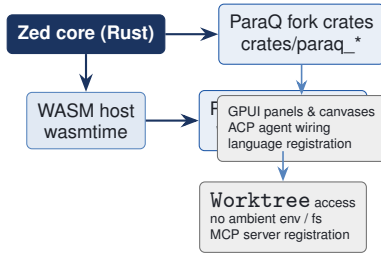


Figure 2. Two integration seams in Zed. Sandboxed WASM extensions (`zed_extension_api`) cover languages, themes, snippets and MCP servers with no fork; panels, GPU canvases and agent wiring live in additive `crates/paraq_*` fork crates. **ParaQ** uses the lightest seam that works.

style protocol over stdio implemented in `acp_thread` and `agent_servers`. External agents (Zed lists Gemini CLI, Claude Agent, Codex, Copilot) run as *separate processes* and connect through an `AgentConnection` trait that defines session lifecycle (`new_session`, `load_session`, `resume_session`, `close_session`) and interaction (`prompt`, `cancel`, `authenticate`), with streaming responses and token-usage reporting. As of recent releases an *ACP Registry* distributes such agents to any ACP-compatible client, and Zed can run several agent threads in parallel.

For **ParaQ** this changes the agent design fundamentally: **ParaQ's Athena orchestrator is implemented as an ACP-compatible agent server**, so it plugs into Zed's existing agent panel, thread history and parallelism through a documented protocol rather than a bespoke UI (Section 9, revised). The heavy quantum reasoning lives in the agent process; the editor only renders ACP threads it already knows how to render.

What the source analysis changes

Three findings reshape the plan. (1) Guppy, themes and *tool* servers ship as *WASM extensions* — no fork. (2) The agent should be an *ACP server*, reusing Zed's agent panel, not a from-scratch chat view. (3) Only true GPU surfaces (circuit/HUGR/-molecule canvases) and the language registration justify fork-level crates. This shrinks the fork delta dramatically and lets most of **ParaQ** be built and shipped independently of the editor binary.

3.6 ADR

ADR-00: Integrate at the lightest seam (extension > ACP > fork)

Decision. For each capability, choose the lightest Zed integration seam that suffices, in order: WASM extension, then ACP agent server, then fork crate.

Context. The source analysis shows three seams with very different maintenance costs against a fast-moving upstream (37k+ commits). **Alternatives.**

(a) *Everything in the fork*: maximal control, maximal drift and merge cost. (b) *Everything as extensions*: zero fork, but WASM cannot host GPU canvases or the deep editor surfaces **ParaQ** needs. **Consequences.** A small, reviewable fork delta plus a portfolio of independently-versioned extensions and one ACP agent; capabilities ship on their own cadence. **Status:** Accepted; supersedes the framing of ADR-01.

4 Component Specification: Zed Fork and GPU Extensions

Persona: quantum engineer, platform engineer. Building on Section 3, the editor core is a fork of Zed whose **GPU** framework provides a hybrid immediate/retained-mode, GPU-accelerated UI in Rust. **ParaQ's** rule (G4, ADR-00) is to add crates, views and registrations rather than patch Zed internals, so the fork tracks upstream with a small, reviewable delta.

4.1 Modifications to the fork

Four additive changes constitute the entire delta against upstream Zed. First, a *language registration* for Guppy that supplies the tree-sitter grammar (Section 6), the LSP adapter and syntax/indent queries. Second, three new *GPU panel views* — the HUGR graph viewer, the Selene emulation panel and the agent chat — each implemented as a `Render` type docked in the workspace. Third, a *molecular viewer* view backed by WGPU. Fourth, a set of *commands* (palette actions) that bridge to the integration services. No upstream file is edited in place; the delta lives in `crates/paraq_*`.

4.2 GPU view contract

A **ParaQ** panel is an ordinary `GPU` view: a state struct plus a `Render` implementation. The emulation panel, for example, holds a handle to the emulation service and renders shot histograms and resource metrics as they stream in.

```

1 use gpui::{div, prelude::*, Context, Render, Window};
2
3 /// GPU view for the Selene emulation panel (L6 surface).
4 pub struct EmulationPanel {
5     service: EmulationServiceHandle, // → L4 Selene service
6     last: Option<EmulationResult>,
7     running: bool,
8 }
9
10 impl Render for EmulationPanel {
11     fn render(&mut self, _w: &mut Window,
12              cx: &mut Context<Self>) → impl IntoElement {
13         div()
14             .flex().flex_col().gap_2().p_3()
15             .child(self.toolbar(cx)) // run / stop / seed
16             / shots
17             .child(match &self.last {
18                 Some(r) => self.histogram(r, cx),
19                 None    => self.placeholder(cx),
20             })
21     }
22 }
  
```



```
21 }
```

4.3 ADR

ADR-01: Fork Zed for the editor core

Decision. Use a fork of Zed (GPUI) as the editor core rather than VS Code/Electron, a CodeMirror/Monaco web editor, or a from-scratch editor. **Context.** ParaQ needs native performance (G6), a Rust-native extension surface that matches the Rust-heavy quantum stack, and a GPU renderer for circuit/graph views. **Alternatives.** (a) *VS Code fork*: largest ecosystem, but Electron’s memory/latency profile and a JS/TS extension surface sit awkwardly against a Rust core. (b) *Monaco in a pure-web shell*: fastest to start, but no native file/process performance and a weaker story for embedding WGPU views. (c) *Build from scratch*: maximal control, prohibitive cost. **Consequences.** We inherit Zed’s velocity and GPUI, accept tracking a fast-moving upstream (mitigated by the additive-delta rule), and standardise on Rust across the core. **Status:** Accepted.

5 Component Specification: Tauri v2 Shell and IPC

Persona: platform engineer. Tauri v2 is the desktop shell: it owns the window, the auto-updater, the OS key-chain bridge and — the reason it is chosen over a bare GPUI binary — a *capability* system that scopes every privileged action. The shell also supervises the Python sidecar that hosts the AI runtime and the heavier quantum/ML services.

5.1 IPC design

Two IPC channels exist (Fig. 7). Front-end (GPUI/WebView) to Rust core uses *Tauri Commands*: typed, capability-gated functions. Rust core to Python sidecar uses *JSON-RPC 2.0* over a length-prefixed stdio pipe (or a localhost UNIX/named-pipe socket), with request/response plus server-initiated notifications for streaming progress.

```
1 use serde::{Deserialize, Serialize};
2
3 /// A Tauri command exposed to the front-end. Capability "
4   quantum:emulate"
5   /// must be granted or the call is rejected by the shell
6   before it runs.
7 #[tauri::command(rename_all = "snake_case")]
8 pub async fn emulate_guppy(
9     state: tauri::State<'_, AppState>,
10    req: EmulateRequest,
11 ) -> Result<EmulateResponse, IpcError> {
12     state.guard.require("quantum:emulate")?; //
13     // capability check
14     state.sidecar.rpc("selene.emulate", req).await // -> JSON-
15     RPC to sidecar
16 }
```

```
13 #[derive(Serialize, Deserialize)]
14 pub struct EmulateRequest {
15     pub hmgr_b64: String, // compiled program
16     pub n_qubits: u32,
17     pub shots: u32,
18     pub seed: u64, // reproducibility (G7)
19     pub error_model: Option<ErrorModelSpec>,
20     pub simulator: SimulatorKind, // Quest | Stim
21 }
```

5.2 JSON-RPC sidecar envelope

Every sidecar method shares one envelope so the bridge is uniform. Notifications carry a *method* but no *id*; the sidecar streams them for progress and the core fans them onto the event bus.

```
1 { "jsonrpc": "2.0", "id": 42,
2   "method": "selene.emulate",
3   "params": { "hmgr_b64": "...", "n_qubits": 4,
4               "shots": 2000, "seed": 12478,
5               "simulator": "stim" } }
```

5.3 ADR

ADR-02: Tauri v2 shell with a Python JSON-RPC sidecar

Decision. Package with Tauri v2 and run Python work (LangGraph, HF inference, pytket/Selene) in a supervised sidecar addressed by JSON-RPC 2.0. **Alternatives.** (a) *PyO3 in-process embedding*: lowest call latency, but a GIL-bound Python interpreter inside the editor process risks blocking the UI (G6) and weakens the sandbox (G5). (b) *gRPC sidecar*: strong typing and streaming, but adds protobuf build complexity and a heavier dependency for a local, single-host link. (c) *REST over HTTP*: simplest, but awkward for bidirectional streaming of agent/emulation events. **Consequences.** Process isolation gives us the security model of Section 13 and keeps Python off the UI thread; we accept serialisation overhead, mitigated by length-prefixed binary framing for large HUGR/array payloads. **Status:** Accepted.

6 Component Specification: Guppy LSP and Tree-sitter Grammar

Persona: quantum engineer. The Guppy language server is what makes the quantum language feel native (G2): completion aware of Guppy’s linear types, diagnostics that surface no-cloning and ownership violations *before* emulation, hovers that explain HUGR constraints, and code lenses that run a kernel on Selene. It implements the Microsoft LSP and is registered with the Zed language registry.

6.1 Tree-sitter grammar

Because Guppy is Python embedded, the grammar extends a Python base with the Guppy-specific surface: the `@guppy` decorator family, the `qubit` and `array[qubit, n]` types, the `@owned` annotation, and the quantum builtins. The grammar drives syntax highlighting, structural selection and folding; semantic analysis is the server's job.

```
1 // tree-sitter-guppy: highlight query excerpt (queries/
  highlights.scm)
2 (decorator (identifier) @function.macro
3   (#match? @function.macro "guppy"))
4 (type (identifier) @type.builtin
5   (#any-of? @type.builtin "qubit" "array" "angle"))
6 (call function: (identifier) @function.builtin
7   (#any-of? @function.builtin "h" "cx" "rz" "measure"))
```

6.2 Server capabilities and the linear-type analyzer

The server advertises completion, hover, diagnostics, document symbols, semantic tokens, code lenses and code actions. Its distinguishing feature is a *linear-type analyzer*: a per-buffer pass that tracks qubit ownership and flags use-after-measure, implicit discard and aliasing as diagnostics keyed to exact ranges — the same guarantees the Guppy compiler enforces, surfaced incrementally as the engineer types.

```
1 from typing import Protocol, Optional
2 from lsprotocol import types as lsp
3
4 class GuppyAnalyzer(Protocol):
5     """Incremental semantic layer over the tree-sitter parse.
6     """
7     def diagnostics(self, uri: str) → list[lsp.Diagnostic]:
8         """Linear-type, ownership and resource diagnostics."""
9         ...
10    def completions(self, uri: str,
11                    pos: lsp.Position) → list[lsp.
12                    CompletionItem]:
13        """Type-aware completions (qubit-valid ops only in
14        scope)."""
15        ...
16    def hover(self, uri: str,
17              pos: lsp.Position) → Optional[lsp.Hover]:
18        """Explain inferred types and HUGR constraints."""
19        ...
20    def code_lenses(self, uri: str) → list[lsp.CodeLens]:
21        """Run on Selene', 'Compile to HUGR', 'Show circuit'.
```

Why the analyzer matters

The cheapest quantum bug is the one caught at the keystroke. By re-implementing Guppy's ownership checks as incremental LSP diagnostics, **ParaQ** rejects use-after-measure and qubit-discard before a single shot is emulated — directly serving correctness-first (G1) and native UX (G2).

6.3 ADR

ADR-03: Out-of-process LSP reusing guppylang

Decision. Implement the Guppy LSP as a Python process that wraps `guppylang`'s own checker for ground-truth diagnostics, with a tree-sitter front-end for fast highlighting/structure. **Alternatives.** (a) *Pure-Rust re-implementation of Guppy semantics*: fastest and in-process, but duplicates a moving upstream type system and risks divergence (violates G4). (b) *Tree-sitter only, no semantic server*: trivial to build but cannot catch linear-type errors — fails G1. **Consequences.** We get authoritative diagnostics that track upstream automatically, at the cost of a process hop; latency is hidden by debouncing and by serving highlights/structure from tree-sitter synchronously. **Status:** Accepted.

7 Component Specification: SELENE Emulation Integration

Persona: quantum engineer, computational scientist. Selene is where most **ParaQ** work happens: it runs compiled Guppy/HUGR programs faithfully — including Helios runtime behaviour such as mid-circuit measurement and feed-forward — with configurable noise and multiple backends, all on a laptop. The integration service wraps **selene-sim** behind the JSON-RPC bridge and streams shots and resource metrics to the emulation panel.

7.1 Emulation service contract

The service exposes a small, stable surface: validate a program, emulate it (ideally or under an error model), and report resource metrics. A fixed seed makes every run reproducible (G7), which is the precondition for putting emulated kernels under CI (Section 19).

```
1 from typing import Protocol, Optional, Literal
2 from pydantic import BaseModel
3
4 class ErrorModelSpec(BaseModel):
5     kind: Literal["none", "depolarizing", "paragon_calibrated"]
6     p_1q: float = 0.0
7     p_2q: float = 0.0
8     seed: Optional[int] = None
9
10 class ResourceMetrics(BaseModel):
11     n_qubits: int
12     gate_count: int
13     two_qubit_gates: int
14     depth: int
15
16 class EmulationResult(BaseModel):
17     counts: dict[str, int] # bitstring → frequency
18     metrics: ResourceMetrics
19     seed: int
20     stack_versions: dict[str, str] # provenance (G7)
21
22 class SeleneService(Protocol):
23     def emulate(self, hugr_b64: str, n_qubits: int, shots: int
24               , seed: int, simulator: Literal["quest", "stim"]
```

```

25     ],
26     error_model: ErrorModelSpec) →
    EmulationResult: ...
    def resources(self, hmgr_b64: str) → ResourceMetrics: ...

```

7.2 The ParagonNoise plugin

Selene cleanly separates the *simulator* (ideal evolution) from the *error model* (all realism) and the *runtime* (system behaviour such as ion transport). **ParaQ** ships a **ParagonNoise** error-model plugin, calibrated to the operating point ParagonCorp targets on Helios-class hardware, so that a “pass” in the validator (Section 9) is a fair rehearsal of a real run. The plugin is registered through **selene-core**; no Selene internals are modified.

7.3 ADR

ADR-04: Wrap Selene as a service; extend via plugins

Decision. Treat Selene as an out-of-process service and extend it only through its documented simulator/error-model/runtime plugin points. **Alternatives.** (a) *Bespoke statevector emulator in Rust*: avoids a Python hop and could exploit GPU’s WGPU, but reimplements Helios runtime semantics that Selene already models faithfully (fails G1/G4). (b) *Hardware only*: no local loop — far too slow and costly for iteration (fails G6). **Consequences.** Free, fast, reproducible local emulation that tracks upstream; the cost is serialising HUGR and large count maps, mitigated by binary framing. **Status:** Accepted.

8 Component Specification: TKET Compiler Pipeline

Persona: quantum engineer, platform engineer. The compile service turns the HUGR emitted by Guppy into a program a target can run, preserving exactly what the circuit computes. It wraps **tket2** (Rust, with the **tket2** Python bindings) and **pytket**, and exposes optimisation level and target as parameters. Figure 3 shows the pipeline the service drives.

8.1 Compile service contract

```

1 from typing import Protocol, Literal
2 from pydantic import BaseModel
3
4 class CompileRequest(BaseModel):
5     guppy_src: str
6     opt_level: Literal[0, 1, 2] = 2
7     target: Literal["helios", "generic"] = "helios"
8     paragon_passes: bool = True # apply paragon.chem
9     rewrites
10
11 class CompileResult(BaseModel):
12     hmgr_b64: str

```

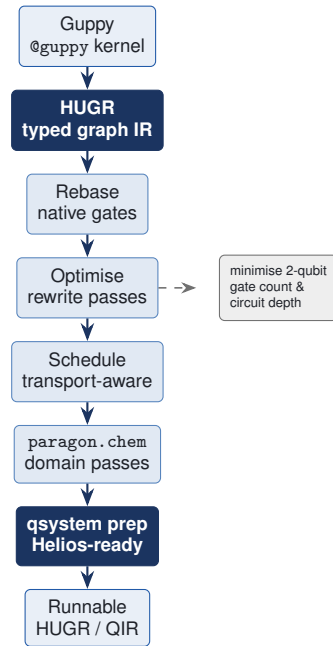


Figure 3. (D3) The Guppy→HUGR→TKET compilation pipeline as driven by the **ParaQ** compile service. Each stage is a HUGR-to-HUGR transform; ParagonCorp’s **paragon.chem** domain passes slot in before the vendor’s **qsystem** preparation, which always runs last.

```

12 mermaid: str # circuit diagram for the
13 viewer
14 metrics_before: dict
15 metrics_after: dict # show the optimisation win
16
17 class CompileService(Protocol):
18     def compile(self, req: CompileRequest) → CompileResult:
19         ...
20     def render_mermaid(self, hmgr_b64: str) → str: ...

```

The service returns the Mermaid string **tket2** already emits (`circ.mermaid_string()`) so the rendering pipeline (Section 16) needs no bespoke circuit renderer, and it reports resource metrics before and after optimisation so the win is visible.

8.2 The **paragon.chem** extension

ParagonCorp defines a **paragon.chem** HUGR extension naming chemistry macro-operations (a fermionic-excitation block, a UCC ansatz layer, a Pauli-measurement-grouping op) and a small set of chemistry-aware rewrite passes that run before **tket2-qsystem** preparation. Because they are ordinary HUGR-to-HUGR passes, they compose with the built-in passes and need no compiler fork; they are validated on Selene by compiling with and without the pass and confirming identical results while resource counts drop.

```

1 use tket2::{Circuit, passes};
2
3 /// Domain pass: fuse UCC rotations and group commuting Pauli
4 /// measurements,
5 /// reducing two-qubit-gate count before the vendor's Helios
6 /// preparation.
7 pub fn paragon_chem_optimise(circ: &mut Circuit) {

```

```

6  passes::apply_greedy_commutation(circ); // reuse tket2
   rewrites
7  paragon::fuse_ucc_rotations(circ); // ParagonCorp-
   specific
8  paragon::group_pauli_measurements(circ); // fewer
   measurement settings
9  }

```

8.3 ADR

ADR-05: Extend HUGR/tket2, never fork the compiler

Decision. Add ParagonCorp optimisation as namespaced HUGR extensions and tket2 passes that slot before `qsystem` prep. **Alternatives.** (a) *Fork tket2*: full control, but guarantees drift from a fast-moving upstream and breaks G4. (b) *Post-process compiled output*: no source access to the IR, brittle and unable to exploit chemistry structure. **Consequences.** Passes compose cleanly and upgrade with the stack; the cost is pinning to a tket2/HUGR version and treating upstream renames (e.g. the `qsystem` rename) as managed dependency upgrades. **Status:** Accepted.

9 Component Specification: Agentic AI Subsystem

Persona: computational scientist. This is the layer that lets a scientist brief **ParaQ** like a capable colleague. It is a LangGraph state graph of cooperating agents that turns a paper, a prompt or a molecular input into a designed experiment, generated Guppy, an emulated result and a cited report — with a human approval gate before any scarce or irreversible action (G3).

9.1 The agent state machine

Figure 4 is the agent graph. A planner decomposes the brief; a retriever grounds it in the literature; a pre-screen step calls foundation models; a code-gen agent emits Guppy from validated templates; a validator type-checks, emulates and compares to a classical baseline, looping back to repair on failure; and a reporter writes the cited result. The single human gate guards the optional hardware run.

9.2 Agent roles and tool schemas

Each agent is a node with a typed tool schema. Table 3 lists the roles; the planner's tools, for instance, are `retrieve`, `pre_screen`, `generate`, `validate` and `report`, each a JSON-schema'd function the LangGraph runtime dispatches.

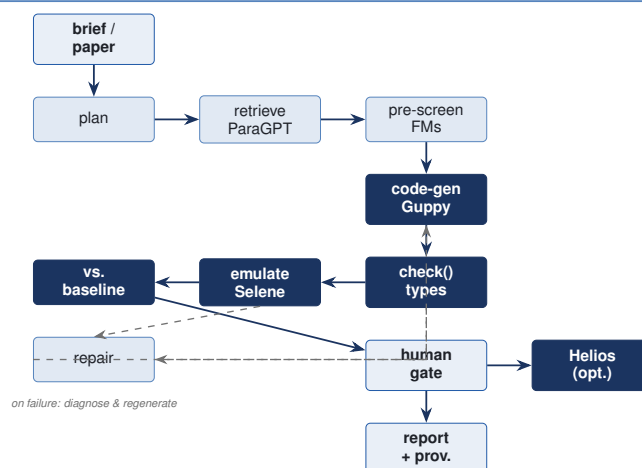


Figure 4. (D2) The agentic workflow as a state machine. Generation is cheap; nothing is trusted until it passes a type check, a Selene emulation and a comparison to a classical baseline. The dashed path is the repair loop; the human gate guards the single paid/irreversible step (hardware).

9.3 Memory

The subsystem keeps two memory tiers. *Episodic* memory is the per-session graph state (the plan, intermediate artefacts, tool results), checkpointed by the LangGraph runtime so a session can resume or be inspected. *Semantic* memory is the durable vector index behind the retriever — the literature and internal corpus — plus a registry of validated Guppy templates the code-gen agent reuses.

9.4 Declarative experiments

An experiment is declared, not coded. The orchestrator reads the declaration, fills the agents from Table 3, and runs the loop of Fig. 4.

```

1  # paraq/experiment.yaml -- a declarative agentic experiment
2  name: antioxidant-groundstate-screen
3  goal: "Rank three candidate antioxidants by ground-state
   stability"
4  retriever: { sources: [internal-rnd, journals, patents], top_k
   : 8 }
5  pre_screen: { chemistry_model: mace-off, keep_top: 2 }
6  quantum:
7    method: vqe
8    active_space: { electrons: 4, orbitals: 4 }
9    backend: selene # validate on emulator first
10 validation: { classical_baseline: ccscd, tolerance_hartree:
   0.0016 }
11 report: { audience: formulation-science, cite: true }

```

9.5 LangGraph node signature

```

1  from typing import TypedDict, Annotated
2  from langgraph.graph import StateGraph
3
4  class ParaQState(TypedDict):
5      brief: str
6      plan: list[str]
7      passages: list[dict] # retriever output (with
   citations)
8      candidates: list[dict] # pre-screen ranking
9      guppy_src: str

```

Agent roles in the ParaQ subsystem

Planner	Decomposes the brief into a typed plan; selects tools, models and quantum method; re-plans on failure.
Retriever	RAG over papers, patents and internal reports; returns grounded passages with citations.
Pre-screen	Calls chemistry/biology/physics foundation models to rank candidates before any quantum spend.
Code-gen	Instantiates validated Guppy templates from a structured spec; never writes from a blank page.
Validator	<code>guppy.check()</code> → Selene emulate → classical-baseline compare; repairs or rejects.
Reporter	Translates results into scientific language; writes a cited, reproducible report.

Table 3. The cooperating agent roles. A human supervises and approves at defined gates; the agent never books hardware on its own.

```

10 validation: dict           # check + emulate + baseline
   verdict
11 approved: bool           # set only at the human gate
12
13 def build_graph() → StateGraph:
14     g = StateGraph(ParaQState)
15     for node in ("plan", "retrieve", "pre_screen",
16                 "code_gen", "validate", "report"):
17         g.add_node(node, AGENTS[node])
18     g.add_edge("plan", "retrieve"); g.add_edge("retrieve", "pre_screen")
19     g.add_edge("pre_screen", "code_gen"); g.add_edge("code_gen", "validate")
20     g.add_conditional_edges("validate",
21                             lambda s: "report" if s["validation"]["passed"] else "code_gen")
22     return g

```

Why this is safe to automate

Three properties make automatic Guppy generation responsible: Guppy rejects ill-typed or resource-unsafe programs at `check()` time; Selene runs the program deterministically under a fixed seed before any hardware is touched; and a classical baseline bounds the acceptable answer. The human approves at the gate — the agent never books hardware on its own (G1, G3).

9.6 ADR

ADR-06: LangGraph for orchestration

Decision. Build the subsystem on LangGraph (explicit state graph) rather than AutoGen (conversational multi-agent) or a bespoke loop. **Alternatives.** (a) *AutoGen*: fast for free-form agent chat,

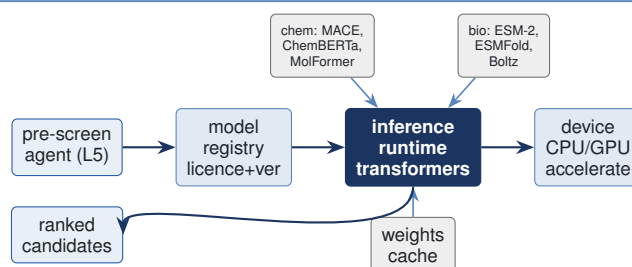


Figure 5. (D5) HuggingFace model integration dataflow. The pre-screen agent calls the sidecar through the model registry (which enforces licence and version); the runtime loads cached weights and runs on the available device. Out-of-scope or unlicensed models are refused at the registry.

but its implicit control flow is harder to make inspectable and reproducible (tension with G1/G7). (b) *Bespoke orchestrator*: maximal control, but reinvents checkpointing, retries and tool routing. **Consequences.** An explicit graph gives deterministic edges, checkpointed state for provenance, and a single place to enforce the human gate; we accept a dependency on LangGraph's evolving API, pinned in Section 17. **Status:** Accepted.

10 Component Specification: HuggingFace Inference Sidecar

Persona: computational scientist, platform engineer. The pre-screen agent needs domain foundation models; the inference sidecar hosts them off the UI thread, behind the same JSON-RPC bridge, with a model registry that records each model's licence and version (a hard governance requirement). Figure 5 shows the dataflow.

10.1 Model registry and inference contract

```

1 from typing import Protocol, Literal
2 from pydantic import BaseModel
3
4 class ModelCard(BaseModel):
5     id: str                # e.g. "facebook/
   esm2_t33_650M_UR50D"
6     domain: Literal["chemistry", "biology", "physics"]
7     task: str              # "embedding" | "structure"
   | "potential"
8     licence: str           # enforced before use
9     revision: str          # pinned commit/tag (
   provenance)
10    commercial_ok: bool
11
12 class InferenceService(Protocol):
13     def list_models(self) → list[ModelCard]: ...
14     def infer(self, model_id: str, inputs: dict,
15              revision: str) → dict:
16         """Refuses if model is absent, unpinned, or out of
17         licence scope."""
18         ...

```


Domain	Models	Role in ParaQ
Chemistry	MACE-MP/OFF, ChemBERTa, MolFormer, DeepChem	Relax conformers, rank stability, seed active-space hints.
Biology	ESM-2, ESMFold, ProteinBERT, Boltz, AlphaFold3 (API)	Characterise skin/hair targets; predict structure & affinity.
Physics	EquiformerV2, MACE-MP, NequIP	Interatomic potentials for materials & dynamics.

Table 4. Foundation-model coverage in the inference sidecar. The model registry records licence, revision and commercial-use status for each entry.

10.2 Domain model coverage

Table 4 maps the model families the sidecar serves to the ParagonCorp question they inform. Chemistry MLIPs relax geometries and rank candidates cheaply; biology models characterise targets and estimate binding; physics potentials support materials and dynamics. All are pinned and licence-checked (Section 17).

Licensing is a governance requirement

Several of the strongest protein models are released under non-commercial licences, and some (e.g. AlphaFold3) are API-gated rather than weight-available. The registry records each model's licence and revision, and the inference service *refuses* any model used outside its licensed scope. This is enforced in code, not left to convention.

11 Component Specification: Molecular I/O Pipeline

Persona: computational scientist. Chemistry and biology enter **ParaQ** as molecular inputs — SMILES strings, PDB structures, .xyz geometries. The molecular pipeline parses, validates and normalises these into a canonical internal form that both the molecular viewer (WGPU) and the quantum chemistry path (active space → Hamiltonian → Guppy) consume.

11.1 Pipeline stages and contract

Inputs are parsed (RDKit for SMILES/MOL, a PDB parser for structures, a plain reader for .xyz), sanitised, optionally relaxed by an MLIP from the sidecar (Section 10), and emitted as a canonical `Molecule` record carrying atoms, bonds, coordinates and provenance.

```
1 from typing import Protocol, Literal
2 from pydantic import BaseModel
```

```
3
4 class Atom(BaseModel):
5     element: str
6     xyz: tuple[float, float, float]
7
8 class Molecule(BaseModel):
9     atoms: list[Atom]
10    bonds: list[tuple[int, int, int]] # (i, j, order)
11    charge: int = 0
12    multiplicity: int = 1
13    source_format: Literal["smiles", "pdb", "xyz", "mol"]
14    provenance: dict # hash, parser version (G7)
15
16 class MolecularIO(Protocol):
17     def parse(self, data: str,
18              fmt: Literal["smiles", "pdb", "xyz", "mol"]) →
19         Molecule: ...
20     def relax(self, mol: Molecule, model_id: str) → Molecule:
21         # via MLIP
22         ...
23     def to_active_space(self, mol: Molecule,
24                        electrons: int, orbitals: int) →
25         dict:
26             """Hand-off to the chemistry path (InQuanto/Guppy)."""
27         ...
```

11.2 ADR

ADR-07: RDKit-centred parsing with MLIP relaxation

Decision. Use RDKit as the primary chemical parser/sanitiser and relax geometries with an MLIP from the inference sidecar before the quantum step. **Alternatives.** (a) *Hand-rolled parsers:* avoid a heavy dependency but re-implement decades of edge-case handling. (b) *Classical force-field relaxation only:* cheaper, but MLIPs give better starting geometries that improve VQE convergence. **Consequences.** Robust parsing and good initial geometries at the cost of an RDKit dependency (permissively licensed) in the sidecar. **Status:** Accepted.

12 Data Models and API Contracts

This section consolidates the cross-cutting contracts: the IPC command surface, the JSON-RPC method catalogue, and the core Rust traits that bind the integration services together. The per-component types in Sections 7, 8, 10 and 11 are the payloads these contracts carry.

12.1 Tauri command surface

Table 5 is the complete front-end command surface. Every command is capability-gated (Section 13); a command whose capability is not granted is rejected by the shell before its body runs.

12.2 JSON-RPC method catalogue

The sidecar exposes a flat, versioned method namespace. Requests carry an `id`; streaming progress arrives as `id-`

Command	Capability	Effect
compile_guppy	quantum:compile	Guppy → HUGR (+ metrics, mermaid).
emulate_guppy	quantum:emulate	Run on Selene; stream shots.
resources	quantum:compile	Resource metrics only.
agent_run	agent:run	Start an experiment graph.
agent_approve	agent:approve	Satisfy the human gate.
infer	model:infer	Foundation-model inference.
parse_molecule	mol:parse	Parse SMILES/PDB/xyz.
submit_hardware	hw:submit	Queue a Helios job (gated).

Table 5. The complete Tauri command surface and the capability each requires. Paid or irreversible commands (submit_hardware) require an explicit, separately granted capability and a satisfied human gate.

less notifications.

```

1 // namespaces: selene.* tket.* agent.* model.* mol.*
2 "tket.compile" // → { hmgr_b64, mermaid, metrics_* }
3 "selene.emulate" // → { counts, metrics, seed, versions }
4 "selene.resources" // → ResourceMetrics
5 "agent.run" // streams agent.progress notifications
6 "agent.approve" // resolves a pending human gate
7 "model.infer" // → task-specific payload
8 "mol.parse" // → Molecule
9 // notifications (no id):
10 "agent.progress" "selene.shot" "compile.metric"

```

12.3 Core Rust traits

The core process binds services behind small traits so the sidecar transport is swappable and each service is independently testable.

```

1 use async_trait::async_trait;
2
3 #[async_trait]
4 pub trait QuantumService: Send + Sync {
5     async fn compile(&self, req: CompileRequest)
6         → Result<CompileResult, ServiceError>;
7     async fn emulate(&self, req: EmulateRequest)
8         → Result<EmulateResponse, ServiceError>;
9 }
10
11 #[async_trait]
12 pub trait AgentService: Send + Sync {
13     /// Start a run; progress is delivered via the event bus.
14     async fn run(&self, spec: ExperimentSpec)
15         → Result<RunHandle, ServiceError>;
16     async fn approve(&self, run: RunHandle, gate: GateId)
17         → Result<(), ServiceError>;
18 }
19
20 /// One uniform error taxonomy across IPC and services.
21 #[derive(thiserror::Error, Debug)]
22 pub enum ServiceError {
23     #[error("capability denied: {0}")] Denied(String),
24     #[error("sidecar transport: {0}")] Transport(String),

```

Zone	Trust, holdings and limits
Core (Rust)	Trusted. Holds capability grants and the keychain handle; holds no model weights. Mediates all privileged actions.
Sidecar (Py)	Semi-trusted, sandboxed. Holds models and runs heavy compute; holds no secrets; network via allowlist only.
Front-end	Untrusted. May call only granted, capability-scoped commands; cannot reach the filesystem, network or secrets directly.

Table 6. The three trust zones. Untrusted content (a parsed paper, a model output, a web page) executes only in the front-end/sidecar and can never escalate its own capabilities.

```

25 #[error("validation failed: {0}")] Validation(String),
26 #[error("upstream tool error: {0}")] Upstream(String),
27 }

```

12.4 OpenAPI surface for CI

For the platform persona, the same operations are exposed by a local REST facade (loopback only, token-gated) so CI can drive compile/emulate without the GUI. The facade mirrors the JSON-RPC catalogue one-to-one.

```

1 openapi: 3.1.0
2 info: { title: ParaQ Local API, version: "1.0" }
3 paths:
4   /v1/compile:
5     post:
6       operationId: compile
7       requestBody: { $ref: '#/components/CompileRequest' }
8       responses: { "200": { description: CompileResult } }
9   /v1/emulate:
10    post:
11      operationId: emulate
12      responses: { "200": { description: EmulationResult } }

```

13 Security and Sandboxing Model

Persona: platform engineer. Security is by construction (G5). **ParaQ** handles untrusted inputs at every turn — parsed PDFs, model completions, web content, molecular files from collaborators — and the model’s job is to ensure none of them can widen their own privileges or exfiltrate secrets.

13.1 Trust zones

Table 6 states the three zones and what each may and may not do. The invariant: privilege flows *down* the trust order only, and never the reverse.

13.2 Capability model

Tauri v2 capabilities are declared per window and per command; **ParaQ** ships the *least-privilege* default set and treats paid or irreversible actions as separately-granted, gated capabilities.

Threat	Control
Prompt injection in a parsed paper	Retrieved text is quoted as data; tool calls require schema-valid arguments; no tool is invoked from document text alone.
Agent over-reach	Human gate before any paid/irreversible action; hw:submit and fs:write are not default capabilities.
Secret exfiltration	Secrets only in keychain, injected per-request by the core; sidecar has no secret access; egress allowlisted.
Supply-chain drift	Pinned versions (Section 17); checksum-verified weights; CI rebuilds against each upstream bump.
Malicious molecular file	Parsed and sanitised by RDKit in the sandbox; no code execution from file content.

Table 7. Security threats specific to an agentic, document-ingesting IDE and the control that contains each.

```

1 // src-tauri/capabilities/default.json
2 { "identifier": "paraq-default",
3   "windows": ["main"],
4   "permissions": [
5     "core:event:default",
6     "paraq:quantum-compile", "paraq:quantum-emulate",
7     "paraq:agent-run", "paraq:model-infer", "paraq:mol-parse"
8   ] } // NOTE: hw:submit and fs:write are NOT in the
        default set

```

13.3 Sandboxing the sidecar

The Python sidecar runs with a restricted profile: no inherited environment secrets, a working directory confined to the workspace, and outbound network limited to an allowlist (HuggingFace model hosts, the Nexus endpoint, configured arXiv mirrors). All other egress is denied. Secrets (Nexus tokens, model-hub tokens) live only in the OS keychain and are injected by the core per-request, never written to disk or passed through the front-end.

13.4 Prompt-injection and untrusted-content defence

Because the agent ingests papers and web content, it treats all such content as data, never instructions. Table 7 lists the principal threats and the control that contains each.

ADR-08: Capability-scoped, sandboxed sidecar over in-process Python

Decision. Run untrusted/heavy work in a sandboxed sidecar behind Tauri capabilities rather than embedding Python in the core. **Alternatives.** (a) *In-process (PyO3)*: fastest, but a compromise

Operation	Target	Method
Keystroke-to-glyph	<16 ms (60 fps)	GPU frame timing.
LSP completion	<120 ms p95	Debounced; tree-sitter sync.
LSP diagnostics	<300 ms p95	Incremental analyzer.
HUGR compile (small)	<1.5 s	tkt2 wall-clock.
Selene emulate (4q, 2k shots)	<2 s	Stim backend.
Model infer (embedding)	<500 ms	Warm cache, GPU.
Agent step over-head	<200 ms	Graph transition, excl. tool.

Table 8. Performance budgets, measured continuously in CI on reference hardware. UI-thread operations are latency-bound; service operations are throughput/turnaround-bound.

in any Python dependency would run with the core's privileges. (b) *Full container/VM per sidecar*: strongest isolation, but heavy for a desktop app and awkward for GPU passthrough. **Consequences.** Process-level isolation plus a capability gate is a strong, practical boundary; we accept IPC cost and a coarser-than-VM sandbox, hardened by the egress allowlist and keychain-only secrets. **Status:** Accepted.

14 Performance Targets and Benchmarking Plan

Persona: platform engineer. Performance is a goal (G6) but never at the expense of correctness (G1). The targets below are budgets the team commits to and measures in CI; Fig. 6 illustrates the qualitative split of where time is expected to go in a typical agentic run.

14.1 Latency and throughput budgets

Table 8 states the budgets. Editor interactions must feel instantaneous; LSP feedback must keep pace with typing; heavy work (emulation, inference) runs off the UI thread and is judged on throughput and turnaround, not frame latency.

14.2 Benchmarking harness

The harness is a fixed suite of reference workloads — a Bell-state smoke test, a 2-qubit VQE step, an H₂ ground-

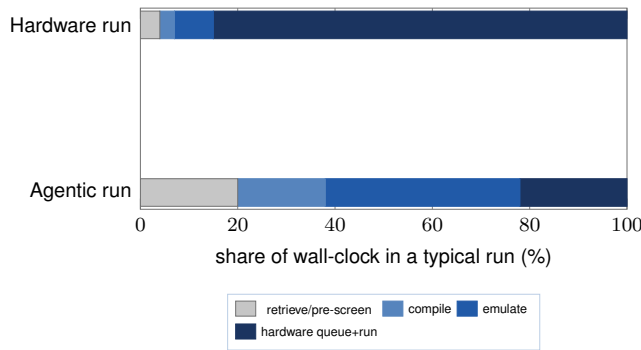


Figure 6. Illustrative split of wall-clock time across an agentic run (emulator path) versus a run that includes a hardware job (schematic, not measured). On the emulator path the time is dominated by emulation and model calls; once hardware is involved, the shared queue dominates — which is why most iteration stays local.

state run, an embedding inference, and a full agentic experiment — each run with deterministic seeds so results are comparable across commits. The harness records latency percentiles and resource metrics and fails CI on regression beyond a tolerance.

```
1 import time
2 from paraq.bench import case, percentile
3
4 @case(name="vqe_step", warmups=3, runs=30)
5 def bench_vqe_step(ctx):
6     t0 = time.perf_counter()
7     ctx.compile("vqe_ansatz.guppy", opt_level=2)
8     ctx.emulate(n_qubits=4, shots=2000, seed=12478, simulator="stim")
9     return time.perf_counter() - t0 # harness aggregates p50 /p95
```

14.3 Reproducibility as a performance property

Because every emulation is seeded and every artefact versioned (G7), a benchmark is not only fast-or-slow but *stable*: the same workload must produce the same counts and resource profile across runs. Treating reproducibility as a measured property is what lets **ParaQ** put quantum programs under ordinary CI.

15 Component Interaction and Workflow

The contracts above compose into two end-to-end flows worth drawing explicitly: how the processes talk during a single emulation (Fig. 7), and how an engineer’s paper-to-simulation journey crosses the personas and services (Fig. 8).

15.1 Component interaction (sequence)

15.2 Paper-to-simulation journey (swimlane)

16 Circuit and Graph Visualisation Pipeline

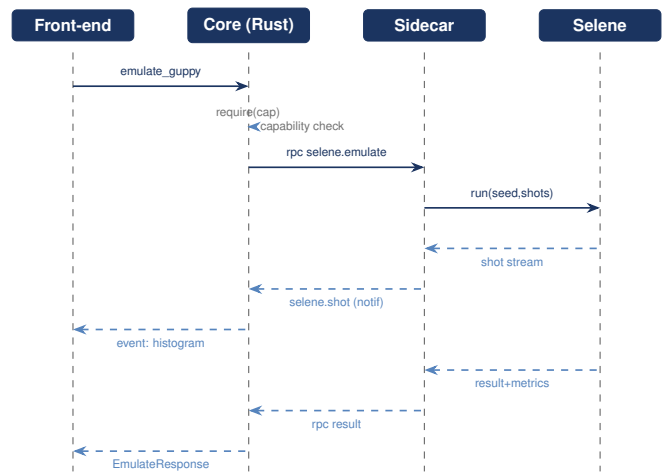


Figure 7. (D4) Component-interaction sequence for one emulation. The front-end calls a capability-gated Tauri command; the core checks the capability and relays JSON-RPC to the sandboxed sidecar; Selene streams shots back, which the core fans onto the event bus as the histogram updates live.

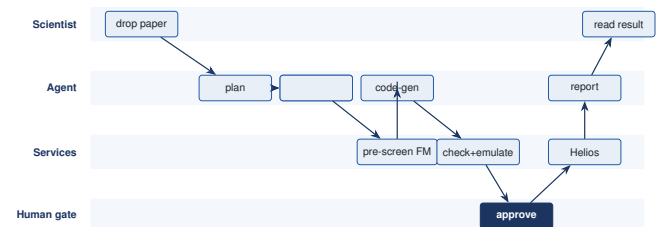


Figure 8. (D7) The paper-to-simulation journey as a swimlane across four roles. The scientist only drops a paper and reads the result; the agent and services do the work; the single human gate authorises the scarce hardware step before the reporter writes up a cited, reproducible answer.

Persona: quantum engineer. **ParaQ** shows engineers their circuits and HUGR graphs without a bespoke renderer where possible. Figure 9 is the rendering pipeline: tket2 emits a Mermaid string for circuits, HUGR is laid out with a graph engine, and both are drawn through GPUI/WGPU; D3.js is available in embedded WebView panels for interactive graph exploration.

16.1 Rendering contract

```
1 /// A laid-out, GPUI-drawable circuit/graph model.
2 pub struct RenderModel { pub nodes: Vec<RNode>, pub edges: Vec
  <REdge> }
3 pub trait Visualiser {
4     /// Build a render model from compiled output.
5     fn from_hugr(&self, hugr_b64: &str) -> RenderModel;
6     /// Reuse tket2's Mermaid for circuit-shaped graphs.
7     fn mermaid(&self, hugr_b64: &str) -> String;
8 }
```

The rule mirrors the architecture: the viewer is a *shell over upstream output*, not a re-implementation. When an engineer asks “show me the circuit,” the panel renders tket2’s Mermaid; when they ask “did it work,” it shows Selene’s resource metrics and the validator’s verdict.

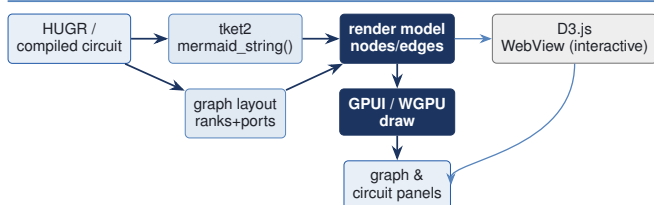


Figure 9. (D6) The visualisation rendering pipeline. Circuits reuse the Mermaid string tkt2 already produces; HUGR graphs are laid out (rank/port assignment) and both feed a render model drawn natively through GPU/WGPU, with an optional D3.js interactive view in a WebView panel.

17 Compatibility Matrix and Pinned Versions

Persona: platform engineer. Every external dependency is pinned. The matrices below are the build contract; CI rebuilds against each upstream bump and treats a break as a managed upgrade, never a silent drift (G4). Versions are a snapshot at authoring time and must be re-verified against Section 20.

17.1 Build and packaging

The workspace is built with Cargo (Rust) and uv or Poetry (Python), with an optional Nix flake for hermetic developer environments. Tauri bundles the signed desktop artefacts per platform; the Python sidecar ships as a frozen environment alongside the binary.

```

1 # Cargo.toml (workspace excerpt) -- additive ParaQ crates only
2 [workspace]
3 members = ["crates/paraq_app", "crates/paraq_services",
4           "crates/paraq_lsp", "crates/paraq_viz",
5           "crates/paragon_passes"] # tkt2/hugr extension
6
7 [workspace.dependencies]
8 tauri      = { version = "2", features = ["protocol-asset"] }
9 serde     = { version = "1", features = ["derive"] }
10 async-trait = "0.1"
11 thiserror  = "1"

```

18 ParaQ Capability Development Plan

Persona: all. This section turns the architecture (Sections 2 and 3) into a concrete, buildable programme. It is written to be executed incrementally with Claude Code: each *workstream* below names the crates and files it creates, the Zed integration seam it uses (per ADR-00), the Quantinuum component it binds to, the typed interface it must satisfy, and an acceptance test. The ambition is explicit — to make **ParaQ** the most advanced AI-augmented quantum IDE, *starting from* the Quantinuum stack — and the plan is deliberately structured so that the differentiating capabilities (document-to-circuit, multimodal synthesis, agentic validation, GPU circuit/HUGR canvases) are reachable in increments, each shippable and testable on Selene without hardware.

Component	Pinned	Notes
Rust toolchain	1.95.0 (stable)	Per Zed <code>rust-toolchain.toml</code> ; profile minimal.
Tauri	2.x	v2 capability model required.
Zed (fork base)	tracked	Additive delta; re-based per upstream tag.
GPU	via Zed	Not a separate crate pin.
zed_extension_api	0.6.0	WASM extension ABI; check Zed compat.
WASM target	wasm32-wasip2	Extension compile target.
ACP	ProtocolVersion::V1	Min. version for the Athena agent.
Python	3.11-3.12	Sidecar interpreter.
guppylang	≥0.21	Bundles Selene; LSP ground truth.
selene-sim	matched	To guppylang; QuEST/Stim back-ends.
tkt2 / hugr	pinned pair	Renames are managed upgrades.
pytket	current	Bridge for legacy circuits.
LangGraph	pinned	Orchestration graph.
transformers	pinned	HF inference runtime.
RDKit	pinned	Molecular parsing.

Table 9. Build-time version pins for the **ParaQ** stack. The pair (tkt2, hugr) must be upgraded together because the IR and compiler co-evolve.

18.1 Capability-to-seam map

Every capability is assigned the lightest seam that suffices (Table 10). This is the single most important planning artefact: it determines what ships as an independently-versioned WASM extension, what ships as the ACP agent, and what requires fork crates.

Figure 10 shows the same decomposition as a dependency layering, making clear that the WASM extensions and the ACP agent are buildable and releasable independently of the editor binary, while only four fork crates carry GPU surfaces.

Capability	Seam (ADR-00)	Quantinuum binding	Where it lives
Guppy syntax, highlight, indent	WASM extension	tree-sitter-guppy	extensions/guppy
Guppy LSP (types, HUGR hints)	WASM ext. + sidecar	guppylang, hugr	extensions/guppy + paraq_lsp
Selene emulation panel	Fork crate (GPU)	selene-sim	crates/paraq_selene
HUGR graph canvas	Fork crate (GPU/GPU)	hugr	crates/paraq_hugr
Circuit canvas	Fork crate (GPU/GPU)	tkt2 (mermaid/render)	crates/paraq_circuit
Molecule viewer (SMILES/PDB/xyz)	Fork crate (GPU)	-- (RDKit sidecar)	crates/paraq_mol
TKET compile actions	Sidecar + actions	pytket, tkt2	paraq_compile svc
Athena orchestrator	ACP agent server	all + HF	paraq_athena (proc)
Document-to-circuit	ACP agent tool	guppylang + HF	paraq_athena tools
Multimodal synthesis	ACP agent tool	guppylang + HF	paraq_athena tools
HF model inference	Python sidecar	transformers	paraq_infer svc
Context-aware completion	WASM ext. + agent	guppylang	extensions/guppy

Table 10. Capability-to-seam map. Each **ParaQ** capability is assigned the lightest Zed integration seam that suffices (ADR-00) and bound to a Quantinuum component. Only four paraq_* fork crates carry GPU surfaces.

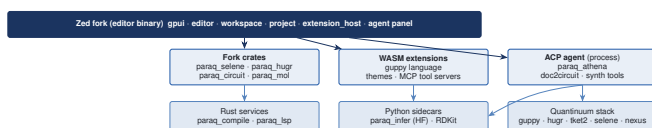


Figure 10. The development decomposition. Fork crates carry GPU surfaces; WASM extensions and the ACP agent ship independently; services and sidecars bind to the Quantinuum stack and HuggingFace. Arrows are build/runtime dependencies.

18.2 Workstream A — Guppy language extension (WASM)

Goal. First-class Guppy editing: syntax, highlighting, indentation, brackets, and an LSP adapter, shipped as a sandboxed Zed extension that requires no fork.

Seam. WASM extension via `zed_extension_api`.
Creates. `extensions/guppy/` with `extension.toml`, `languages/guppy/{config.toml, highlights.scm, injections.scm}`, a Tree-sitter grammar `tree-sitter-guppy`, and `src/guppy.rs` implementing the Extension trait to launch the Guppy LSP.

Because Guppy *is* Python with decorators and a linear type discipline, the grammar derives from Python and adds Guppy keywords (`guppy`, `qubit`, `owned`, `comptime`, `measure`). The LSP adapter is the only procedural code, and it must respect the WASM sandbox — using the API's `Worktree` to locate the interpreter rather than ambient environment access.

```

1 use zed_extension_api::{self as zed, LanguageServerId,
2   Worktree, Command, Result};
3
4 struct GuppyExtension;
5
6 impl zed::Extension for GuppyExtension {
7   fn new() → Self { GuppyExtension }
8
9   fn language_server_command(
10     &mut self,
11     _id: &LanguageServerId,
12     worktree: &Worktree,
13     // sandbox-safe host
14     access
15   ) → Result<Command> {
16     // Locate the paraq guppy LSP on PATH via the worktree,
17     // never std::env.
18     let path = worktree
19       .which("paraq-guppy-lsp")
20       .ok_or_else(|| "paraq-guppy-lsp not found on PATH"
21         .to_string());
22     Ok(Command { command: path, args: vec!["--stdio"].into
23       (), env: vec![] })
24   }
25 }

```



```

19 }
20
21 zed::register_extension!(GuppyExtension);

```

Acceptance. Install as a dev extension; open a .guppy file; verify highlighting, that the LSP starts, and that a deliberate linear-type violation (reusing a measured qubit) surfaces a diagnostic. **Claude Code tasks.** “Generate the Tree-sitter grammar from the Python grammar plus this keyword list”; “write `highlights.scm` covering decorators and quantum builtins”; “scaffold the `Extension` impl and a smoke test”.

18.3 Workstream B — Guppy LSP and HUGR-aware analysis

Goal. A language server that goes beyond Python: it type-checks via `guppylang`, surfaces linear-type/ownership errors inline, and offers HUGR-aware hover and completion. **Seam.** Native Rust service (`crates/paraq_lsp`) spawned by Workstream A; heavy checks delegated to a Python worker that calls `guppy.check()` and `.compile()`. **Interface.** Standard LSP over stdio; internally a typed bridge:

```

1  /// Bridge from the LSP server to the Python guppy worker.
2  pub trait GuppyAnalysis: Send + Sync {
3      /// Type-check a source buffer; map guppylang diagnostics
4      /// to LSP.
5      fn check(&self, uri: &str, src: &str) -> anyhow::Result<
6          Vec<Diagnostic>>;
7      /// Compile to HUGR and return a summary (node/edge counts,
8      /// qubit width).
9      fn compile_summary(&self, uri: &str, src: &str) -> anyhow
10         ::Result<HugrSummary>;
11      /// Resource estimate for the current program (for the
12      /// gutter badge).
13      fn resources(&self, uri: &str, src: &str) -> anyhow::
14         Result<ResourceMetrics>;
15 }
16
17 pub struct HugrSummary { pub nodes: u32, pub edges: u32, pub
18     qubits: u32 }
19 pub struct ResourceMetrics { pub qubits: u32, pub
20     two_qubit_gates: u32, pub depth: u32 }

```

Acceptance. On the teleportation example, `check` returns zero diagnostics and `compile_summary` reports the expected qubit width; a mutated buffer that drops a qubit yields a precise diagnostic with the offending range. **Claude Code tasks.** “Implement the JSON-RPC worker protocol between `paraq_lsp` and the Python `guppy` worker”; “map `guppylang` error objects to LSP `Diagnostics` with ranges”; “add a debounce so checks run on idle, on the background executor”.

18.4 Workstream C — Selene emulation panel (fork crate)

Goal. A dockable GPU panel that runs the active program on Selene (ideal and noisy), streams shot histograms and resource metrics, and pins a random seed for reproducibility. **Seam.** Fork crate `crates/paraq_selene` (GPU view + entity); emulation runs off-thread on the `BackgroundExecutor` and reports back by updating the entity. **Interface.**

```

1  /// L4 service contract the panel (L6 view) talks to.
2  /// Implemented over a
3  /// Python sidecar that drives selene-sim.
4  #[async_trait::async_trait]
5  pub trait EmulationService: Send + Sync {
6      async fn run(&self, req: EmulationRequest) -> anyhow::
7         Result<EmulationResult>;
8  }
9
10 pub struct EmulationRequest {
11     pub hugr: HugrProgram,           // compiled program
12     pub n_qubits: u32,
13     pub shots: u32,
14     pub backend: Backend,           // Quest | Stim
15     pub error_model: Option<NoiseSpec>,
16     pub seed: u64,                 // reproducibility (G7)
17 }
18
19 pub struct EmulationResult {
20     pub counts: Vec<(String, u64)>, // bitstring -> frequency
21     pub metrics: ResourceMetrics,
22     pub log: String,
23 }
24
25 pub enum Backend { Quest, Stim }

```

Acceptance. Running the Bell program yields counts clustered on 00/11; enabling the depolarizing model introduces a small fraction of 01/10; identical seeds reproduce identical counts; the UI stays responsive (no frame drops) during a 16–20 qubit run. **Claude Code tasks.** “Implement the `EmulationPanel` `Render` with a histogram element”; “wire `cx.spawn` to call the sidecar and update the entity on completion”; “add seed/shots/backend controls bound to actions”.

18.5 Workstream D — HUGR and circuit GPU canvases (fork crates)

Goal. Two interactive, GPU-accelerated canvases: a HUGR graph viewer (hierarchical, collapsible regions) and a circuit diagram. **Seam.** Fork crates `crates/paraq_hugr` and `crates/paraq_circuit`, each a custom GPU `Element` for virtualised, pannable/-zoomable rendering at 120 fps. **Quantinuum binding.** HUGR is consumed from `guppy.compile()`; circuits can be rendered from `tket2`’s Mermaid emission as a fallback while the native canvas matures. **Interface (rendering model).**

```

1  /// Immutable layout the GPU element draws; produced off-
2  /// thread from a HUGR.
3  pub struct CircuitLayout {
4      pub wires: Vec<Wire>,           // qubit/classical lines
5      pub gates: Vec<GateGlyph>,      // box/control/target
6      pub measures: Vec<MeasureGlyph>,
7      pub width_cols: u32,
8  }
9
10 pub trait LayoutEngine {
11     fn layout_circuit(&self, hugr: &HugrProgram) -> anyhow::
12         Result<CircuitLayout>;
13     fn layout_hugr(&self, hugr: &HugrProgram) -> anyhow::
14         Result<GraphLayout>;
15 }

```

Acceptance. A 20-gate circuit renders correctly and pans/zooms without jank; collapsing a HUGR region hides its children and preserves edges across the boundary; selecting a gate highlights the source span in the editor (bidirectional mapping). **Claude Code tasks.** “Implement a column-assignment layout for circuits from

a HUGR dataflow region”; “write the GPU Element with hit-testing for selection”; “bridge selection to the editor buffer via the project API”.

18.6 Workstream E — Athena as an ACP agent server

Goal. The agentic brain — planning, retrieval, code-gen, validation, reporting — exposed through the Agent Client Protocol so it appears in Zed’s existing agent panel with thread history and parallel threads. **Seam.** ACP agent server process `paraq_athena` implementing the protocol Zed’s `AgentConnection` expects; *no* bespoke chat UI. **Interface.** The agent process implements the ACP session/interaction surface:

```
1  /// ACP-facing surface ParaQ's Athena implements (mirrors Zed's
2  s AgentConnection).
3  #[async_trait::async_trait]
4  pub trait AcpAgent: Send + Sync {
5      async fn new_session(&self, cwd: PathBuf) → Result<
6          SessionId>;
7      async fn load_session(&self, id: SessionId) → Result<()>;
8      async fn resume_session(&self, id: SessionId) → Result<()>;
9      async fn close_session(&self, id: SessionId) → Result<()>;
10     /// Streaming prompt/response; yields content + tool-call
11     events.
12     async fn prompt(&self, id: SessionId, msg: PromptMsg, tx:
13         EventSink) → Result<()>;
14     async fn cancel(&self, id: SessionId) → Result<()>;
15     async fn authenticate(&self, method: AuthMethod) → Result<()>;
16 }

```

Internally Athena runs the plan-act-observe loop over the tools of Workstreams F–G, calling the compile/emulate/infer services. Because it is a separate process, the heavy Python (LangGraph, transformers, guppylang) never blocks the editor, and the agent can be upgraded independently. **Acceptance.** From the Zed agent panel, “new ParaQ thread” connects; a prompt streams a plan and tool calls; two ParaQ threads run in parallel; cancellation is honoured. **Claude Code tasks.** “Scaffold an ACP server in Python using the agent-client-protocol schema”; “implement session lifecycle and a streaming prompt”; “register tools and emit tool-call events over ACP”.

18.7 Workstream F — Document-to-circuit generation

Goal. Parse an arXiv paper (PDF/LaTeX), a book chapter, or a structured brief, and emit an annotated, type-checked Guppy program that reproduces a described circuit or algorithm. **Seam.** An Athena tool (Workstream E) backed by the HF sidecar (parsing/embedding) and guppylang (validation). **Pipeline.** extract → identify circuit/algorithm → draft Guppy from validated templates → `check()` → emulate → self-repair on failure. **Tool schema.**

```
1  {
2      "name": "document_to_circuit",
3      "description": "Generate a type-checked Guppy program from

```

```

4      a document.",
5      "input_schema": {
6          "type": "object",
7          "properties": {
8              "source": {"type": "string", "description": "path or
9                  arXiv id"},
10             "target": {"enum": ["circuit", "vqe", "qpe", "custom"]},
11             "max_qubits": {"type": "integer"},
12             "validate": {"type": "boolean", "default": true}
13         },
14         "required": ["source"]
15     }
16 }

```

Acceptance. Given a paper describing the Bell/teleportation circuit, the tool returns Guppy that type-checks and reproduces the expected distribution on Selene; a generation that fails `check()` is repaired or rejected, never surfaced as “done”. **Claude Code tasks.** “Implement a PDF/LaTeX extractor that yields a structured circuit spec”; “map the spec onto Guppy templates with the validation gate of Fig. 4”; “add a self-repair step that feeds `check()` errors back to the model”.

18.8 Workstream G — Multimodal synthesis and HF inference

Goal. Accept natural language, pseudocode, and molecular inputs (SMILES, PDB, .xyz) and synthesise chemistry/biology-targeted Guppy, seeded by foundation-model pre-screening. **Seam.** Athena tools plus the Python inference sidecar `paraq_infer`. **Quantinuum/HF binding.** MLIPs (MACE family) relax geometry and rank candidates; protein models (ESM/Boltz) characterise targets; the quantum step (VQE via InQuanto+Guppy) refines only the strongly-correlated core. **Interface (sidecar).**

```

1  from typing import Protocol
2
3  class InferenceService(Protocol):
4      def embed(self, text: str, model: str) → list[float]: ...
5      def relax_geometry(self, xyz: str, model: str = "mace-off")
6        → str: ...
7      def rank_candidates(self, smiles: list[str], model: str)
8        → list[float]: ...
9      def predict_structure(self, sequence: str, model: str = "
10         esmfold") → str: ...

```

Acceptance. A SMILES input is relaxed and ranked; only the top candidate is handed to a generated VQE program; the model registry refuses a model used outside its licence (a governance check, not optional). **Claude Code tasks.** “Stand up the FastAPI sidecar exposing `InferenceService`”; “add a model registry with licence metadata and a refusal path”; “wire the pre-screen tool so quantum runs only on survivors”.

18.9 Workstream H — TKET compile integration

Goal. Compile Guppy/HUGR through tket2 with the H-series (qsystem) preparation, exposed as palette actions and an Athena tool, with a circuit-diff view showing two-qubit-gate reduction. **Seam.** A Rust service `paraq_compile` over a Python worker (pytket/tket2) plus

Milestone	Workstreams	Demonstrable outcome
M1 Edit	A, B	Guppy editing with live type-/linearity diagnostics.
M2 Run	C, H	Compile + emulate from the editor; reproducible histograms.
M3 See	D	Interactive circuit and HUGR canvases bound to the buffer.
M4 Augment	E, F, G	Athena ACP agent: doc-to-circuit and multimodal synthesis with the validation gate.

Table 11. The four delivery milestones, each demonstrable on Selene without hardware. Build order is M1→M4; the local edit/check/emulate loop and the agent are front-loaded because they unlock the differentiating capabilities.

GPUI actions. **Acceptance.** Compiling a chemistry ansatz reduces entangling-gate count versus the unoptimised circuit; the qsystem pass runs last; the diff view shows before/after metrics. **Claude Code tasks.** “Expose a `compile` action that calls the worker and opens the circuit canvas on the result”; “surface pass-by-pass resource metrics”; “pin the `tket2/qsystem` version and add a CI check against upstream renames”.

18.10 Sequencing and milestones

The workstreams compose into four milestones, each independently demonstrable on Selene (Table 11). The ordering front-loads the local loop (edit → check → emulate) and the agent, because those unlock the differentiating capabilities and need no hardware.

How to drive this with Claude Code

Each workstream is scoped to a crate or service with a typed interface and an acceptance test, so it maps to a Claude Code session: point it at the named path, give it the interface and the test, and let it implement against the seam. Build order M1→M4; keep the fork delta to the four `paraq_*` crates of Table 10; ship the Guppy extension and the Athena agent on their own cadence. The validation gate (Fig. 4) and reproducible seeds (G7) are non-negotiable in CI — they are what make AI-generated quantum code trustworthy.

19 Development Roadmap

Persona: all. The programme is phased to deliver value early and defer the scarce, expensive parts (hardware) until the local loop is solid. Each phase has an explicit exit so the team can stop, slow or accelerate on evidence.



Figure 11. The four-phase ParaQ roadmap. Timeframes are indicative; each phase has a clear exit gate so scope can be cut or extended on evidence.

Phase	Key deliverables	Exit criteria
P1 Shell (0–3 mo)	Zed fork, Tauri shell, IPC bridge, sidecar supervisor, editor panels.	A signed build that edits files and round-trips a JSON-RPC call.
P2 Quantum (3–9 mo)	Guppy LSP, tree-sitter grammar, TKET compile, Selene emulate, graph/circuit viewer.	Author → compile → emulate a Bell/H ₂ kernel in-IDE.
P3 Agentic (9–18 mo)	LangGraph agents, ParaGPT RAG, FM sidecar, molecular pipeline, declarative experiments.	A paper → validated emulated result with a cited report.
P4 Hardware (18 mo+)	Nexus submission, gated hardware runs, calibrated ParagonNoise, CI benchmarking.	A validated pilot run on Helios, benchmarked vs. classical.

Table 12. Phase deliverables and exit criteria. “Not yet” is an acceptable, documented outcome at any gate.

19.1 Risks and mitigations

The roadmap in one sentence

Build the shell, make the quantum stack feel native, layer agentic AI with a hard validation gate, and reserve scarce hardware for the final, approved, benchmarked step — extending the open Quantinuum stack throughout rather than forking it.

Risk		Mitigation
Upstream drift (Zed/Tauri/Quantinuum)		Additive deltas, pinned versions, CI against each bump; never fork.
Agent hallucination		Hard validation gate (types + emulate + baseline); citations; human approval.
Model licensing breach		Registry with licence/revision; inference refuses out-of-scope models.
Performance regressions		CI benchmark harness with per-op budgets and tolerances.
Security escalation		Capability model, sandboxed sidecar, keychain-only secrets, egress allowlist.
Over-claiming advantage		Maturity labels and mandatory classical baselines in every report.

Table 13. Principal programme risks and the architectural choices that contain them.

20 References

All URLs were verified at the time of writing. Treat versions and capability claims as a snapshot and re-check against the live sources.

- [1] Zed Industries, *Zed editor and GPUI framework*. <https://github.com/zed-industries/zed>
- [2] Tauri, *Tauri v2 documentation — application shell, capabilities and IPC*. <https://v2.tauri.app/>
- [3] Quantinuum, *Open-source software stack (GitHub organisation)*. <https://github.com/Quantinuum>
- [4] Quantinuum, *guppylang — the Guppy quantum programming language*. <https://github.com/Quantinuum/guppylang>
- [5] Quantinuum, *Guppy documentation*. <https://docs.quantinuum.com/guppy/>
- [6] Quantinuum, *hugr — Hierarchical Unified Graph Representation*. <https://github.com/Quantinuum/hugr>
- [7] M. Koch, A. Borgna, S. Sivarajah *et al.*, “HUGR: A Quantum-Classical Intermediate Representation,” arXiv:2510.11420, 2025. <https://arxiv.org/abs/2510.11420>
- [8] Quantinuum, *tket2 — the TKET quantum compiler*. <https://github.com/Quantinuum/tket2>
- [9] Quantinuum, *tket2 Rust crate documentation*. <https://docs.rs/tket2/latest/tket2/>
- [10] Quantinuum, *tket2-hseries / qsystem extension*. https://docs.rs/tket2-hseries/latest/tket2_hseries/
- [11] Quantinuum, *selene — the Selene emulator*. <https://github.com/Quantinuum/selene>
- [12] Quantinuum, *Selene documentation*. https://docs.quantinuum.com/selene/user_guide/overview.html
- [13] Quantinuum, *Selene ErrorModel API reference*. https://docs.quantinuum.com/selene/api/generated/selene_core.error_model.ErrorModel.html
- [14] Quantinuum, *Nexus — cloud platform*. <https://www.quantinuum.com/blog/announcing-the-launch-of-quantinuum-nexus-our-all-in-one-quantum-computing-platform>
- [15] M. Koch *et al.*, “Guppy: Pythonic Quantum-Classical Programming,” PPlanQC, 2025. <https://ks.cs.uchicago.edu/publication/guppy-planqc/guppy-planqc.pdf>
- [16] Microsoft, *Language Server Protocol Specification (3.17)*. <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>
- [17] tree-sitter, *Incremental parsing system*. <https://tree-sitter.github.io/tree-sitter/>
- [18] LangChain, *LangGraph — stateful multi-agent orchestration*. <https://langchain-ai.github.io/langgraph/>
- [19] Microsoft, *AutoGen — multi-agent conversation framework*. <https://microsoft.github.io/autogen/>
- [20] HuggingFace, *Transformers, Diffusers and Accelerate*. <https://huggingface.co/docs>
- [21] I. Batatia *et al.*, “A foundation model for atomistic materials chemistry” (MACE), *J. Chem. Phys.* **163**(18):184110, 2025. <https://doi.org/10.1063/5.0297006>
- [22] T. Hayes *et al.*, “Simulating 500 million years of evolution with a language model” (ESM-3), *Science*, 2025. <https://www.science.org/doi/10.1126/science.ads0018>
- [23] J. Wohlwend *et al.*, “Boltz-2: Exploring the Frontiers of Biomolecular Prediction,” 2025. <https://github.com/jwohlwend/boltz>
- [24] RDKit, *Open-source cheminformatics*. <https://www.rdkit.org/>
- [25] Quantinuum, *InQuanto — computational chemistry platform*. <https://www.quantinuum.com/products-solutions/inquanto>
- [26] Zed Industries, *GPUI framework README* (entities, views, elements; hybrid immediate/retained mode; Metal/WGSL). <https://github.com/zed-industries/zed/blob/main/crates/gpui/README.md>
- [27] Zed Industries, *rust-toolchain.toml* (Rust 1.95.0; targets wasm32-wasip2, wasm32-unknown-unknown, x86_64-unknown-linux-musl). <https://github.com/zed-industries/zed/blob/main/rust-toolchain.toml>
- [28] Zed Industries, *The Zed Rust Extension API* (`zed_extension_api` 0.6.0; `Extension trait`; `register_extension!`). https://github.com/zed-industries/zed/blob/main/crates/extension_api/README.md
- [29] Zed Industries, *Developing Extensions* (manifest, WASM sandbox, languages/themes/MCP servers). <https://zed.dev/docs/extensions/developing-extensions>
- [30] Zed Industries, *External Agents and the Agent Client Protocol (ACP)*. <https://zed.dev/docs/ai/external-agents>
- [31] Zed Industries, *Rust coding guidelines* (`.rules` / `AGENTS.md`; GPUI context, entity and window model). <https://github.com/zed-industries/zed/blob/main/.rules>